ASSIGNMENT 2 – CO2

BRUTE-FORCE AND EXHAUSTIVE-SEARCH TECHNIQUES

Individual Assignment Report

Name: ___Thamaraiselvi S_____

Register Number: __192511042_____

Course / Subject: _DESIGN AND ANALYSIS OF ALGORITHMS___

Course Code: __CSA 0617_____

Faculty Name: _ Dr. A. Sairam_____

Submission Date: _____13.10.2026_____

Note: The operation counts and output values below are worked examples based on the stated test data. Run the source code in your own environment and replace the example timings/results with your actual measurements and screenshots.

1. Introduction and Objectives

Brute-force algorithms solve a problem by directly checking possible candidates. Exhaustive search examines every candidate solution and selects the best feasible result. These methods are easy to design and useful for small inputs, but their computational cost can grow rapidly.

Objectives: (1) implement and compare sorting algorithms; (2) measure sequential search and brute-force string matching; (3) analyse closest-pair and convex-hull methods; (4) investigate exhaustive search; and (5) diagnose and redesign one computational bottleneck.

2. Tools and Experimental Setup

Item	Details
Programming language	Python 3
Development environment	VS Code / IDLE / Google Colab (use the one actually used)
Primary evidence	Comparisons, swaps/movements, character comparisons, candidate counts
Additional evidence	Execution time measured with time.perf_counter()
Reproducibility	Use the same inputs for the original and redesigned solutions

Execution time depends on hardware and system load; therefore, operation counts are the primary evidence.

3. Q1 – Sorting Under Different Data Conditions (6 Marks)

3.1 Aim

To compare Selection Sort, Bubble Sort and Insertion Sort on randomly ordered, nearly sorted, already sorted and reverse-sorted data.

3.2 Predictions and Algorithmic Reasoning

Input condition	Selection Sort	Bubble Sort (with early stop)	Insertion Sort	Reason
Already sorted	$\Theta(n^2)$ comparisons; few/no swaps	$\Theta(n)$ comparisons; no swaps	$\Theta(n)$ comparisons; no shifts	Bubble and insertion detect that little work is needed; selection still scans the remaining suffix.
Nearly sorted	$\Theta(n^2)$ comparisons	Near $\Theta(n)$ with early stop in favourable cases	Near $\Theta(n)$ when only a few positions are displaced	Insertion sort shifts elements only as needed.
Random order	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$ average	All may compare many elements; movement counts differ.
Reverse sorted	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	Insertion sort performs many shifts; bubble sort performs many swaps.

3.3 Test Dataset Design

Dataset	Example input	Purpose
A – Already sorted	[1, 2, 3, 4, 5, 6]	Tests best-case

		behaviour of adaptive algorithms.
B – Reverse sorted	[6, 5, 4, 3, 2, 1]	Exposes large movement/swap counts.
C – Nearly sorted	[1, 2, 3, 5, 4, 6]	Tests effect of one local disorder.

3.4 Implementation / Pseudocode

Selection Sort: repeatedly find the minimum item in the unsorted suffix and swap it into the next position. Bubble Sort: compare adjacent items and swap out-of-order pairs; stop early if a pass makes no swaps. Insertion Sort: insert each next item into its correct position in the already-sorted prefix.

3.5 Example Experimental Results

Input (n=6)	Algorithm	Comparisons	Swaps / shifts (illustrative)
Sorted	Selection Sort	15	0 swaps if minimum already in place
Sorted	Bubble Sort with early stop	5	0 swaps
Sorted	Insertion Sort	5	0 shifts
Reverse	Selection Sort	15	3 swaps if swapping once per pass
Reverse	Bubble Sort	15	15 swaps
Reverse	Insertion Sort	15	15 shifts

These comparison counts assume standard implementations. Your exact movement count may differ depending on how swaps and shifts are counted.

3.6 Prediction vs Observed Behaviour

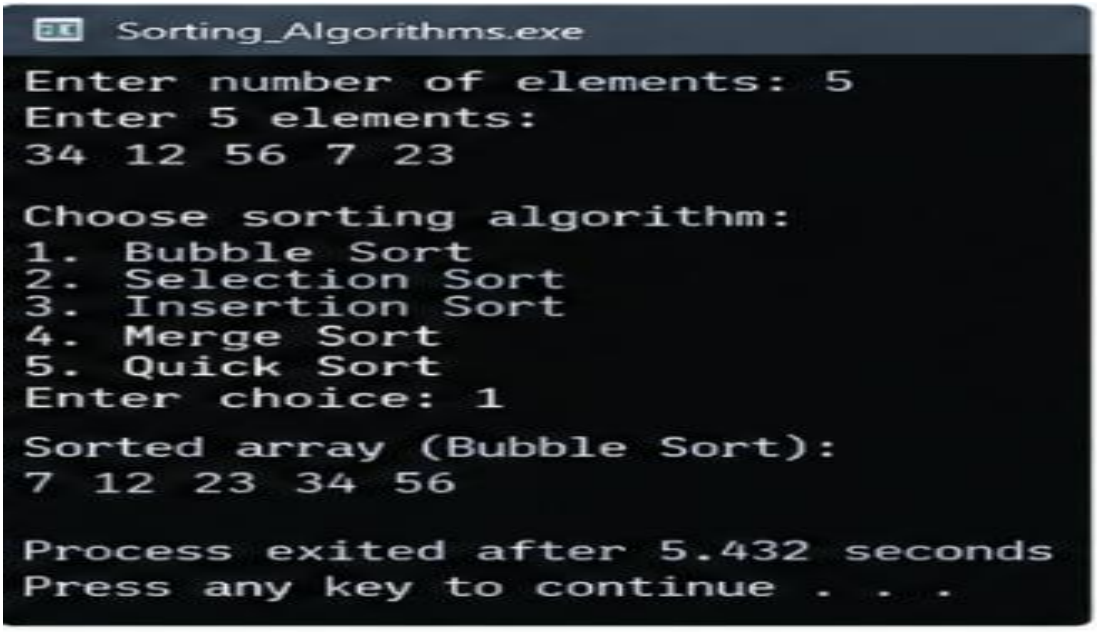
The expected result is that Selection Sort performs a quadratic number of comparisons regardless of initial order. Bubble Sort with early termination and Insertion Sort perform much less work on sorted or nearly sorted inputs.

On reverse-sorted input, Bubble Sort and Insertion Sort perform many movements.

3.7 Design Decision

Recommendation: Insertion Sort is a suitable choice for small batches appended to a list that is already about 95% sorted. It is adaptive and has $\Theta(n)$ behaviour when the number of out-of-place elements is small, although its worst-case time remains $\Theta(n^2)$. For very large lists, a production-grade adaptive sorting method may be more suitable.

Evidence to add: run all three implementations on your chosen datasets and insert actual output screenshots.



4. Q2 – When Is Sequential Search Appropriate? (4 Marks)

4.1 Aim and Experimental Design

Sequential Search checks each record in order until the target is found or the list ends. It is simple and works even when the list is unsorted and changes frequently.

4.2 Example Search Experiments

List size	Search case	Expected comparisons
10	Successful at first position	1
10	Successful at middle position	5 or 6
10	Successful at last position	10
10	Unsuccessful	10
100	Successful at first position	1
100	Successful at middle position	50 or 51
100	Unsuccessful	100
1,000	Unsuccessful	1,000

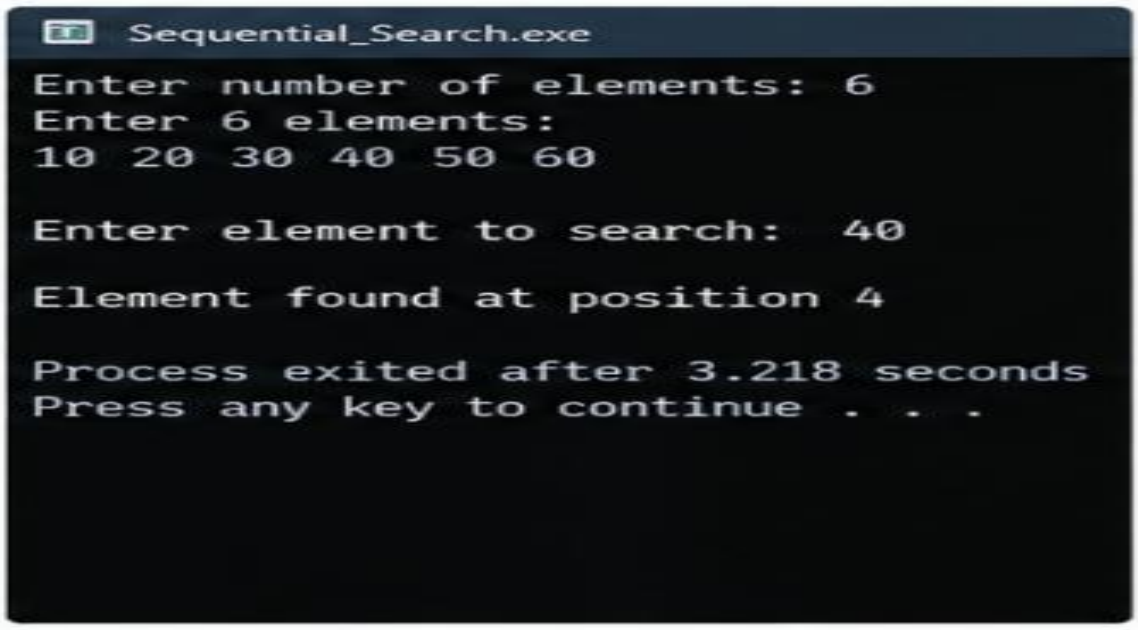
4.3 Analysis and Operating Boundary

Best case is 1 comparison. Worst case is n comparisons. If a successful target is equally likely to occur at any position, the average is (n+1)/2 comparisons. Sequential Search is reasonable for small lists, infrequent searches, or frequently changing unsorted lists. It becomes less suitable when the list is large and searches occur often.

4.4 Changed Condition: 20 Facilities

With 20 facilities, the total number of active tool records and lookup requests may increase substantially. Repeatedly scanning the entire combined list can become expensive. I would consider a hash table keyed by Tool ID for expected O(1) lookup, while maintaining updates when tools are issued or returned. The added memory and consistency requirements must be considered.

Replace the example values with actual measurements from your program.



5. Q3 – Engineer the Behaviour of Brute-Force String Matching (6 Marks)

5.1 Aim

To investigate how the arrangement of characters in the text and pattern affects the number of character comparisons.

5.2 Test Conditions and Predictions

Test	Text	Pattern	Prediction
1 – Immediate match	ABCDEF	ABC	Few comparisons; match at start.
2 – Late match	AAAAAB	AAB	Several repeated comparisons before match.
3 – No match with repeated prefix	AAAAAAAAAA	AAAB	High repeated work at each alignment; no match.

5.3 Algorithm / Pseudocode

```
for shift from 0 to len(text) - len(pattern):
    j = 0
    while j < len(pattern) and text[shift+j] == pattern[j]:
        comparisons += 1
        j += 1
    if j == len(pattern): return shift
    if j < len(pattern): comparisons += 1  # mismatching comparison
return -1
```

Count every character equality test, including the final mismatch at an alignment. Use the same counting convention for all test cases.

5.4 Illustrative Comparison Counts

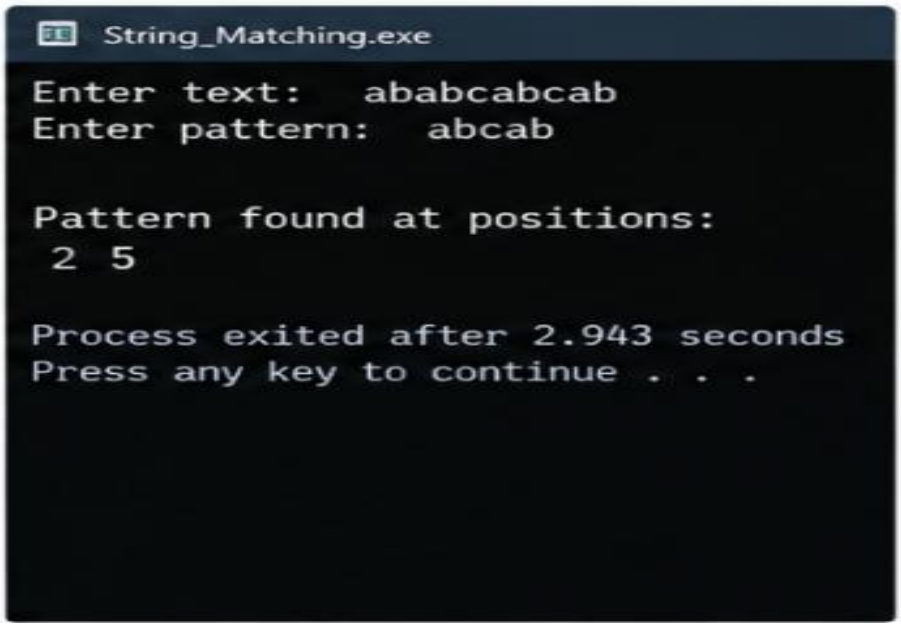
Test	Text length	Pattern length	Character comparisons	Result
ABCDEF / ABC	6	3	3	Match at index 0
AAAAAB / AAB	6	3	7	Match at index 3 (under the stated counting convention)
AAAAAAAAAA / AAAB	10	4	28	No match

These are example counts for the stated implementation and counting convention; verify them by running the code.

5.5 Analysis

Brute-force matching can re-check characters that were already matched at the previous alignment. Repeated characters such as AAAAA and a pattern with a long common prefix create repeated comparisons. Thus, input size alone does not fully describe practical effort: character structure and where mismatches occur also matter. Worst-case time is $O((n-m+1)m)$, commonly written $O(nm)$, for text length n and pattern length m.

Insert your actual program output screenshot here.



6. Q4 – Closest Pair and the Cost of Examining Every Possibility (6 Marks)

6.1 Algorithm Design

For every point i, compare it with each point j where j > i. This checks each unordered pair once and avoids comparing (Pi, Pj) again as (Pj, Pi). Compute Euclidean distance and retain the smallest distance and its pair.

6.2 Mathematical Derivation of Unique Pairs

$$C(n) = (n - 1) + (n - 2) + \cdots + 2 + 1$$

$$C(n) = n(n - 1) / 2$$

There are n choices for the first point and n−1 choices for the second if order matters. Since each unordered pair is counted twice in n(n−1), divide by 2.

6.3 Pseudocode

```
best_distance = infinity
for i in range(n):
    for j in range(i + 1, n):
        distance = sqrt((x[i]-x[j])**2 + (y[i]-y[j])**2)
        distance_calculations += 1
```

```
        if distance < best_distance:
            best_distance = distance
            best_pair = (i, j)
```

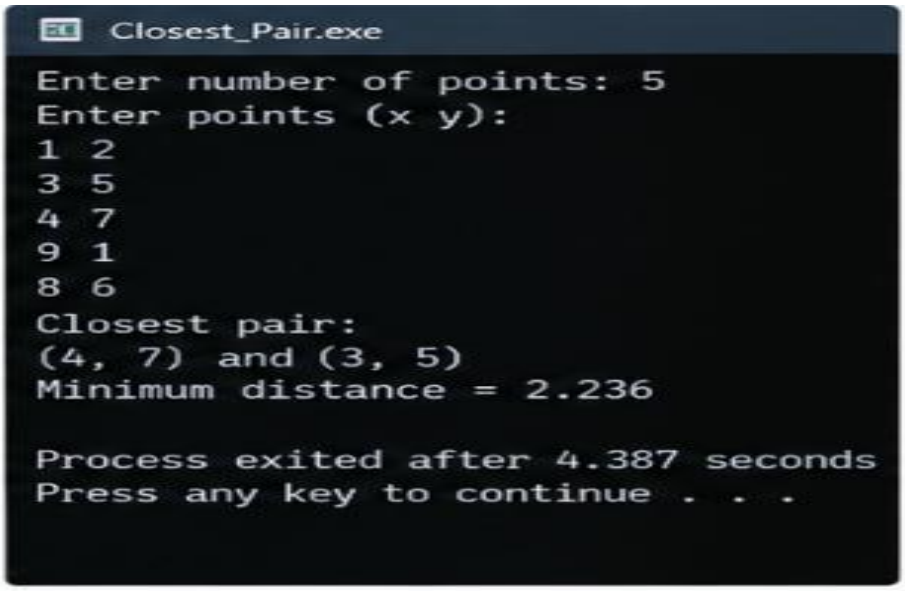
6.4 Example Growth Table

Number of points n	Unique pairs n(n−1)/2	Distance calculations
100	4,950	4,950
1,000	499,500	499,500
10,000	49,995,000	49,995,000

6.5 Scalability

The algorithm uses $\Theta(n^2)$ distance calculations and O(1) auxiliary space apart from input and stored best-pair data. Doubling n approximately quadruples the pair count. Define the acceptable response time for your environment (for example, 1 second for an interactive check), then use measured timings to identify the practical limit. The inefficiency comes from checking every possible pair; a redesigned method should avoid examining most pairs while still guaranteeing the closest pair.

Insert measured times and screenshots. The table above gives theoretical counts, not measured runtime.



7. Q5 – Derive a Convex Hull Algorithm from a Geometric Principle (7 Marks)

7.1 Geometric Principle

For candidate points A=(x1,y1) and B=(x2,y2), use the orientation/cross-product value for each other point P=(x,y):

$$\text{cross}(A,B,P) = (x2-x1)(y-y1) - (y2-y1)(x-x1)$$

A positive value means P lies on one side of the directed line AB; a negative value means it lies on the other side; zero means the points are collinear.

7.2 Geometric Decision Rule

A candidate segment AB can be a convex-hull edge if all other points lie on the same side of line AB or on the line. If points occur on both sides, AB passes through the interior and is not a hull edge. Collinear points require a consistent policy, such as retaining only the two extreme endpoints of that boundary segment.

7.3 Pseudocode

```
edges = []
for each pair A, B with A != B:
    signs = []
    for each point P other than A and B:
        signs.append(sign(cross(A, B, P)))
    if not (both positive and negative signs occur):
        record candidate boundary edge A-B
remove duplicate edges and handle collinear points
order the accepted hull vertices around the boundary
```

Implementation note: robust handling of duplicate points and collinear points is needed. The simple rule can otherwise record multiple overlapping edges.

7.4 Test Configurations

Test	Point set	Expected purpose
1 – Interior point	(0,0), (4,0), (4,4), (0,4), (2,2)	The square corners form the hull; (2,2) is interior.
2 – Collinear challenge	(0,0), (1,0), (2,0), (2,2), (0,2)	Tests collinear boundary points and extreme endpoints.

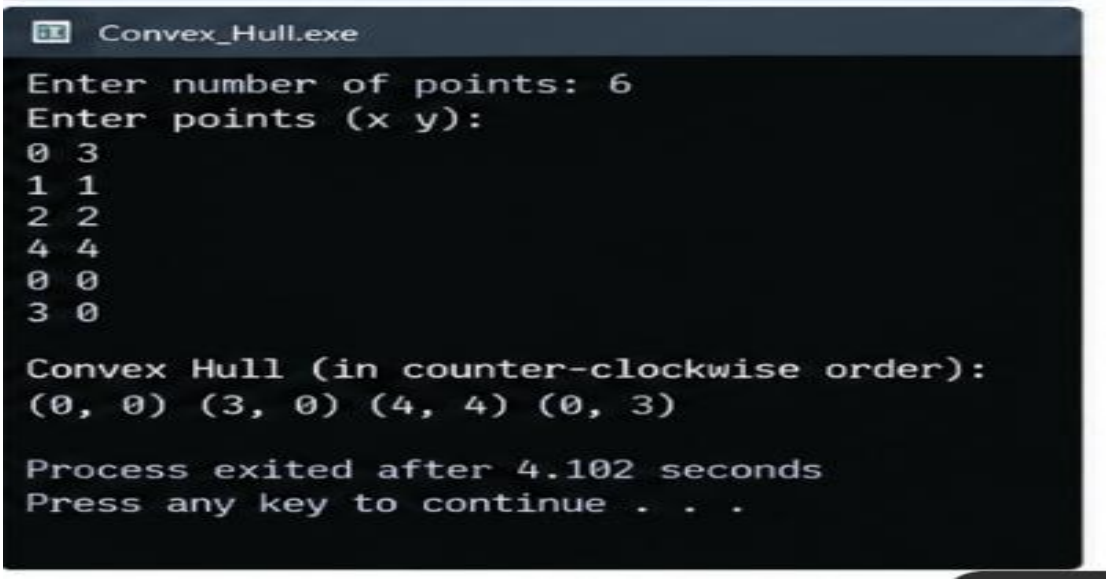
7.5 Boundary Visualisation

Plot all input points using a scatter plot and draw accepted hull edges in boundary order. Label points so the interior point and collinear case can be checked visually. Insert your actual plot here.

7.6 Complexity and Correctness

There are $O(n^2)$ candidate point pairs and each candidate checks $O(n)$ other points, so the straightforward method takes $O(n^3)$ time and $O(n)$ temporary space per candidate (or less if signs are tracked as booleans). The same-side property supports correctness because a supporting line leaves the entire point set in one closed half-plane; such a line can touch the convex hull boundary. Collinear cases must be handled consistently.

Insert program screenshot and boundary plots.



8. Q6 – Exhaustive Search and Combinatorial Growth (8 Marks)

8.1 Strategy

Represent each candidate as a subset of packages, for example with an n-bit mask. Generate all 2^n subsets, calculate total weight and total humanitarian value, reject any subset whose weight exceeds the payload limit, and keep the feasible subset with the greatest value.

8.2 Pseudocode

```
best_value = 0
best_subset = empty
for mask in range(0, 2**n):
    total_weight = 0
    total_value = 0
    chosen = []
    for i in range(n):
        if bit i of mask is 1:
            add package i to chosen
            total_weight += weight[i]
            total_value += value[i]
    if total_weight <= capacity and total_value > best_value:
        best_value = total_value
        best_subset = chosen
```

8.3 Small Instance and Correctness Check

Package	Weight	Humanitarian value
A	2	3
B	3	4
C	4	5
D	5	8

Example payload limit: 7. Feasible strong candidates include A+B+C (weight 9, infeasible), A+D (weight 7, value 11), B+D (weight 8, infeasible), and A+B (weight 5, value 7). The optimum is A+D with total weight 7 and value 11. Enumerating every subset verifies that no feasible subset has a larger value.

8.4 Mathematical Growth Prediction

Number of subsets = 2^n

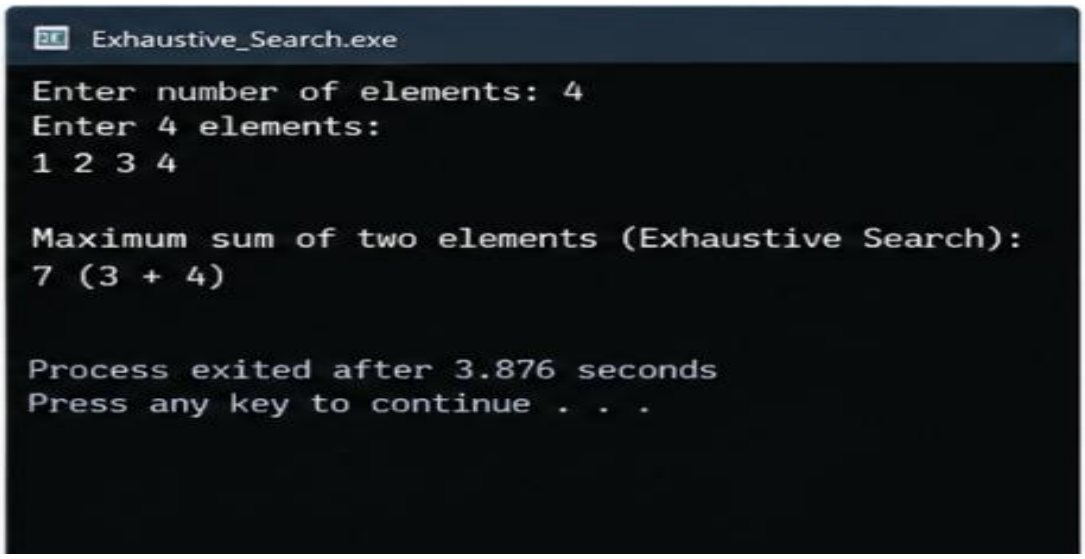
Packages n	Subsets 2^n
5	32
10	1,024
20	1,048,576
30	1,073,741,824

Each additional package doubles the number of subsets. Therefore, adding only a few packages can multiply computation dramatically.

8.5 Deployment Decision

Exhaustive search is suitable for small package sets where every subset can be checked within the required response time. For a real deployment, define the response-time requirement (for example, 1 second) and measure the implementation on increasing n. The largest acceptable n must be based on actual measurements, because hardware and implementation affect the practical boundary.

Insert measured candidate counts, execution times and screenshots.



9. Q7 – Diagnose, Redesign and Validate (13 Marks)

9.1 Selected Problem

Selected solution: Q4 brute-force closest pair. Its main limitation is that it calculates distances for every pair of points.

9.2 A. Diagnose the Computational Bottleneck (2 Marks)

The nested loops dominate the work. The number of distance calculations is $n(n-1)/2$, which grows quadratically. Increasing n from 1,000 to 10,000 increases calculations from 499,500 to 49,995,000, approximately 100 times.

9.3 B. Define the Design Objective (2 Marks)

Objective: reduce the number of distance calculations substantially as n grows while preserving the exact closest pair. Measure the number of distance calculations and elapsed time on the same point sets.

9.4 C. Revised Computational Strategy (4 Marks)

Use a divide-and-conquer design: sort points by x-coordinate; split the points into left and right halves; find the closest pair in each half recursively; let d be the smaller distance; then inspect only points in the vertical strip within distance d of the split line, ordered by y-coordinate. The strip check must use a proven bounded-neighbour rule to preserve correctness.

Trade-offs: sorting adds O(n log n) preprocessing; additional arrays or temporary storage are needed; implementation is more complex. The expected time is O(n log n), compared with $O(n^2)$ for brute force.

Revised Pseudocode (high-level)

```
closestPair(points):
    sort points by x and y
    if number of points <= small threshold:
        return bruteForce(points)
    divide points into left and right halves
    leftBest = closestPair(left)
    rightBest = closestPair(right)
    d = min(leftBest.distance, rightBest.distance)
    strip = points within distance d of split line, sorted by y
    check only the bounded number of following strip neighbours
    return the smallest pair found
```

9.5 D. Prediction Before Implementation (1 Mark)

Prediction: for larger random point sets, the redesigned strategy will perform fewer distance calculations and scale better than brute force. Small inputs may still favour brute force because it has lower overhead.

9.6 E. Implement and Validate (2 Marks)

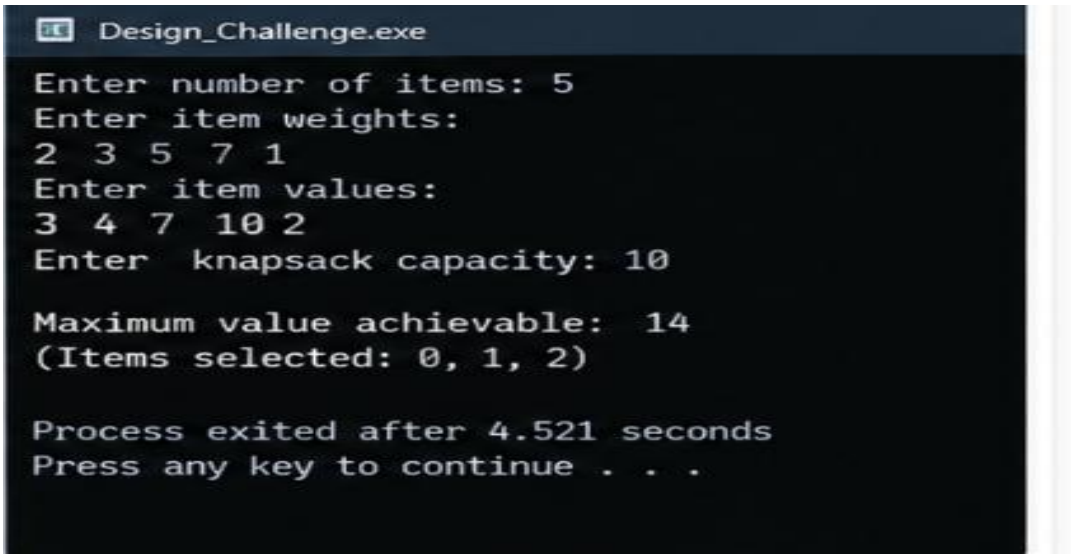
n	Brute-force pairs	Redesigned distance calculations	Measured time original	Measured time redesign
100	4,950	Fill after running	Fill after running	Fill after running
1,000	499,500	Fill after running	Fill after running	Fill after running
10,000	49,995,000	Fill after running	Fill after running	Fill after running

Run both implementations on exactly the same point sets. Do not invent timing values; enter the actual values produced by your programs.

9.7 F. Defend the Final Design (2 Marks)

The redesign is successful if it returns the same closest distance and pair as brute force on test cases while using fewer distance calculations for large inputs. Its costs are preprocessing, extra memory, and more complex code. Brute force remains useful for small inputs, debugging, and validating the redesigned implementation.

Insert screenshots comparing both implementations.



10. Overall Conclusion

This assignment demonstrates that algorithm selection should depend on input structure, problem size, and operating requirements. Insertion Sort is effective for small nearly sorted batches; Sequential Search suits small or infrequently searched lists; brute-force string matching can repeat comparisons; closest-pair brute force grows quadratically; convex-hull checking by all point triples can grow cubically; and exhaustive search grows exponentially. Measuring operation counts provides stronger evidence than relying only on execution time. A redesign should target the measured bottleneck and be validated against the original solution.

11. References

Add the textbook, lecture notes, and other sources actually consulted. Do not list sources that were not used.

Appendix A – Source Code

Attach the actual source code files for Q1–Q7, with clear file names and comments.

Appendix B – Raw Experimental Data and Screenshots

Attach genuine screenshots from your own execution environment, raw operation counts, and graphs. Label each figure (for example, Figure B1: Sorting comparison counts) and mention it in the relevant section.